applied sciences

MDPI

Article

Software Project Risk Identification Based on Scrum Artifacts

Lina Bisikirskienė [1], Lina Čeponienė [1,*], Gytis Vilutis [2] and Adelė Nečionytė [1]

[1] Department of Information Systems, Faculty of Informatics, Kaunas University of Technology, 44249 Kaunas, Lithuania; lina.bisikirskiene@ktu.lt (L.B.); adele.necionyte@gmail.com (A.N.)
[2] Department of Applied Informatics, Faculty of Informatics, Kaunas University of Technology, 44249 Kaunas, Lithuania; gytis.vilutis@ktu.lt
* Correspondence: lina.ceponiene@ktu.lt

Abstract: Risk identification, as a foundational step in risk management, is essential for the success of software development projects, including those using the Scrum methodology. However, risk management in Scrum often lacks structured practices, relying instead on the iterative and collaborative nature of the methodology. This paper presents a novel risk identification framework tailored for Scrum projects, leveraging Scrum artifacts and their associated attributes to enhance risk detection and analysis. The framework encompasses automated data collection from project management environments, such as Jira, thus minimizing the effort required from the project team for risk-related data gathering. By monitoring deviations in artifact attributes and applying predefined thresholds, the framework facilitates the detection of risks at any point in the project lifecycle. Experimental evaluation across three distinct Scrum projects demonstrated the framework's applicability in identifying a wide range of risks, including long-term ones. The proposed framework contributes to Agile project management by offering a structured, automated approach to risk identification, fostering a culture of proactive risk management within Scrum teams.

Keywords: risk management; scrum; agile project management; risk identification

Academic Editor: Asterios Bakolas

Received: 12 December 2024
Revised: 11 January 2025
Accepted: 13 January 2025
Published: 16 January 2025

Citation: Bisikirskienė, L.; Čeponienė, L.; Vilutis, G.; Nečionytė, A. Software Project Risk Identification Based on Scrum Artifacts. *Appl. Sci.* **2025**, *15*, 824. https://doi.org/10.3390/app15020824

1. Introduction

Risk management is the process of identifying, assessing, and mitigating risks that could negatively affect project success [1,2]. In traditional project management methodologies, risk management has well-defined practices, but in Agile methodologies, it is more implicit [3]. Scrum [4] has no formally described project risk management techniques, but, as one of the Agile methodologies, it has natural mechanisms for controlling risks, as it is based on short iterations, timely feedback, and active collaboration. However, these features alone are insufficient for risk identification and further management.

Risk identification is the foundational step of the risk management process, where potential risks are recognized and defined to ensure that they are addressed in subsequent stages [2]. As a continuous process, risks can emerge and be identified at any point in the project lifecycle. This article focuses specifically on the risk identification phase, highlighting its role in fostering risk awareness within the project team and positively influencing the success of the project [5].

Typically, in proposed methodologies, the Scrum process is analyzed as one unit or broken down into sprints for risk management, which can lead to omitting important risks. The Scrum process covers several artifacts that can be analyzed separately to help identify associated risks [4]. The three key Scrum artifacts are the Product Backlog, Sprint Backlog, and Increment. While Scrum emphasizes limited artifact use, in practice, teams

often incorporate additional artifacts such as requirement models, design documents, test plans, etc. [6–8]. The possibility of expanding the list of Scrum artifacts can provide a team with a broader perspective on how to identify project risks related to them [8,9]. Therefore, we propose a hierarchical set of Scrum artifacts that includes the three key artifacts and their sub-artifacts, providing a more detailed perspective on the relationship between artifacts and risks. Furthermore, Scrum artifacts can be linked to track project progress information, which we refer to as attributes (also known as metrics [10] or parameters [11]), which further enhances the risk identification process. The status of an artifact can be monitored through its attribute values, with deviations from expected values signaling potential risk. Project progress information can be manually entered or obtained from the project management environment, enabling calculation of derived values. This process automates risk monitoring and enables timely detection of risks during project execution.

The aim of this paper is to facilitate automated risk identification in Agile software development (Scrum) projects using information from Scrum project artifacts, their attributes, and the proposed project risk classification. One of the primary objectives is to minimize the requirements for human contribution in risk detection, both from the team's and the client's perspectives. This is accomplished by maximizing the automation of project attribute data collection from project management environments, such as Jira (version 10.0.1) [12]. Our approach encompasses risk identification not only within individual sprints but also long-term risk tracking based on attribute values calculated across multiple sprints (e.g., standard deviation of team velocity). We also on risks in software product development, facilitating stakeholder collaboration, team communication, and Scrum ceremonies to facilitate the development process. The proposed methodology also supports the manual entry of attribute values, which is useful when automated data collection is not feasible.

The proposed framework was evaluated over 12 months in three Scrum projects. The results demonstrate its applicability in enabling teams to identify a wide range of risks, including long-term ones.

The remainder of this paper is organized as follows: Section 2 reviews related work in the domain of risk identification and management in Agile projects. Section 3 introduces the proposed risk identification framework and the methodology for its application. Section 4 provides a comprehensive experimental evaluation of the proposed methodology. Finally, Section 5 discusses the results and presents the conclusions.

2. Related Work

Risk management in Agile software development has gained significant attention due to its implications for project success. But while risk management strategies are followed to some extent in Agile projects, they are often implemented in a non-systematic way [13]. Researchers generally agree that incorporating risk management practices into Agile projects can mitigate risks but should be carried out without compromising Agile principles [14,15].

Research on risk management frequently emphasizes the identification and classification of risks into specific categories, such as requirements (requirements elicitation, analysis, specification, change, and other similar risks) [16–20], project management [16–20] (management methods and processes, project initiation, project organization, project planning and control, organizational risks), communication [17–20] (coordination and collaboration, customer collaboration, external stakeholders, group awareness, etc.), technology [16,18,19,21] (environmental risks, infrastructure and resources, security risks, tools selection, technology setup, etc.), design and implementation [19–21], coding and integration [17–19], testing [16–20], humans [16,18–20] (team managing, acquisition and development, trust, performance, work environment, resources, etc.), Agile ceremonies [17–19].

Beyond the principles of risk classification, the risk management process in Agile projects has been analyzed in various studies. These investigations often focus on specific stages of the process, such as risk identification [17,22], risk assessment [15,23], estimation [20], or on the management process in its entirety [3,18,19,21,24–26].

Several proposals have been made to integrate formal project management techniques into the Scrum process for risk management [18,25,26]. The proposal for incorporating the risk management techniques from the Project Management Body of Knowledge Guide [2] into the Scrum process [25] seeks to improve the risk management mechanism in Scrum and increase the success rate of Scrum projects. Another significant contribution is the RIMPRO framework [26], which modifies PMBoK risk management processes to address risks specific to the Product Owner role. RIMPRO covers risk identification and analysis and introduces a classification system tailored to risks associated with the Product Owner. Building on this framework, the authors developed RIMPRO-AST, an automated tool designed to facilitate the management of Product Owner-related risks [27].

In [22], the AGP model is introduced, emphasizing a systematic approach to risk identification. Drawing on 1868 survey responses from professionals, the authors identify four key factors influencing project risks: organizational culture, timeframe and budget stress, project team, and project organizational environment. These factors explain up to 76% of the variability in potential risks. The study highlights the importance of integrating human-centric elements into risk management, emphasizing the collaborative and adaptive nature of Agile methodologies.

In [28], a framework is proposed that integrates machine learning into risk management, introducing a risk prioritization scheme. However, this study primarily focuses on prioritization, with the overall effectiveness of the framework remaining insufficiently validated. Other emerging machine learning solutions for risk management [17,21,23,29] leverage AI-based approaches but are heavily reliant on project datasets. This reliance poses a challenge in Agile environments, where the complexity of collecting high-quality data can hinder the applicability and performance of such methods.

A novel process integrating a recommender system into the Scrum framework to improve risk management in Agile projects is presented in [30]. The system recommends risks and response strategies for projects based on a database of risks from similar past projects. This approach promotes the reuse of software project-related knowledge and aligns with Scrum principles, highlighting the improvement in decision-making and organizational learning in software risk management. Still, the effectiveness of the recommender system again relies heavily on the quality and completeness of stored historical risk data. The authors of [31] introduced the Agile Risk Tool (ART), a lightweight risk management framework designed to minimize human effort by harnessing software agents. ART automates risk identification, assessment, and monitoring using environmental data collected from Agile projects. A significant feature of ART is its ability to dynamically react to changes in project environments through predefined risk rules. Another study [32] built on ART by proposing a goal-driven framework that decomposes the project goals into risk factors and their corresponding indicators. This framework emphasizes integrating risk management into Agile processes by monitoring critical metrics such as team skills, task ownership, and project progress. The model uses autonomous software agents to continuously manage risks, demonstrating effectiveness in case studies.

There are also approaches dedicated to specific aspects of risk management in Agile projects. For example, in [33], an approach is proposed for managing risks in machine learning solution development using Agile methodologies, but it focuses on data and model quality, overlooking the broader scope of risk management. Similarly, authors of [34] present a case study-based approach for risk identification and mitigation in Agile software

re-engineering projects, emphasizing technical debt and legacy system challenges, making it primarily suitable for re-engineering contexts. On the other hand, some approaches target only specific steps in risk management, such as decision-making [35], where fuzzy cognitive maps, mathematical modeling, and expert methods are used to dynamically analyze and mitigate risks. However, the method in Ref. [35] lacks detailed processes for risk identification and classification and does not address integration with Agile practices and tools.

ART [31,32] and similar approaches focus heavily on data-driven risk assessment, but human elements, such as team motivation, interpersonal conflicts, and leadership issues, are not always adequately addressed. These factors, essential in Agile methodologies, are difficult to quantify and can be overlooked by automated systems. Furthermore, integrating the analyzed risk management tools like [30,31] with existing project management and development tools (e.g., Jira) can be challenging, especially when the data structures or formats do not align. Limited access to specific data due to tool compatibility issues can undermine the system's effectiveness in providing comprehensive risk assessments. A more detailed comparison of the analyzed approaches is presented in Table 1. Our proposed framework builds on prior research by combining structured risk classification with automated monitoring of Scrum artifacts and their attributes. This approach aims to address gaps in risk management, such as the lack of formalized processes and the burden of manual data collection, by leveraging automation to improve proactive risk management in Scrum projects.

Table 1. The comparison of the analyzed risk management approaches to our proposed framework (a '+' indicates that the criterion has been met).

Criteria	[22]	[25]	[26,27]	[28]	[30]	[31,32]	This Study
Designed for Agile/Scrum Projects	+	+	+	+	+	+	+
Team participation in risk identification	+	+	+			+	+
Risk classification/categorization/taxonomy	+	+	+	+	+	+	+
Focus on Machine Learning				+	+		
Analyzes risk identification	+	+	+	+	+	+	+
Integration with project management tool (e.g., Jira)			+				+
Threshold-Based risk detection						+	+
Analyzes attributes/artifacts/risks relations	+					+	+
Aligned with Scrum artifacts			+		+	+	+
Implemented in a software tool			+			+	+

3. Risk Identification Methodology for Scrum Projects

The proposed risk identification methodology (Figure 1) consists of two main stages: adapting the identification framework to the specific project and analyzing the risks based on the project data during project execution. Further stages of risk management are not analyzed, as our approach is concentrated on risk identification and ensuring that this process is structured, automated, and aligned with Agile project management and, specifically, Scrum principles.

In the proposed methodology, the terms 'Project Team' and 'Team member' encompass a broader range of participants than the traditional Scrum roles. In addition to the core Scrum roles (Product Owner, Scrum Master, and Development Team), the team may include product managers, product designers, business analysts, DevOps engineers, and

other relevant stakeholders. Any individual who contributes to the project's success is considered a team member. To enhance collaboration and clarify role alignment, the RACI (Responsible, Accountable, Consulted, Informed) matrix [2] can define and allocate responsibilities among team members. Additionally, all team members need to understand the risk identification principles. Active engagement in applying these principles within their areas of responsibility is essential to fostering a risk management culture throughout the project lifecycle.

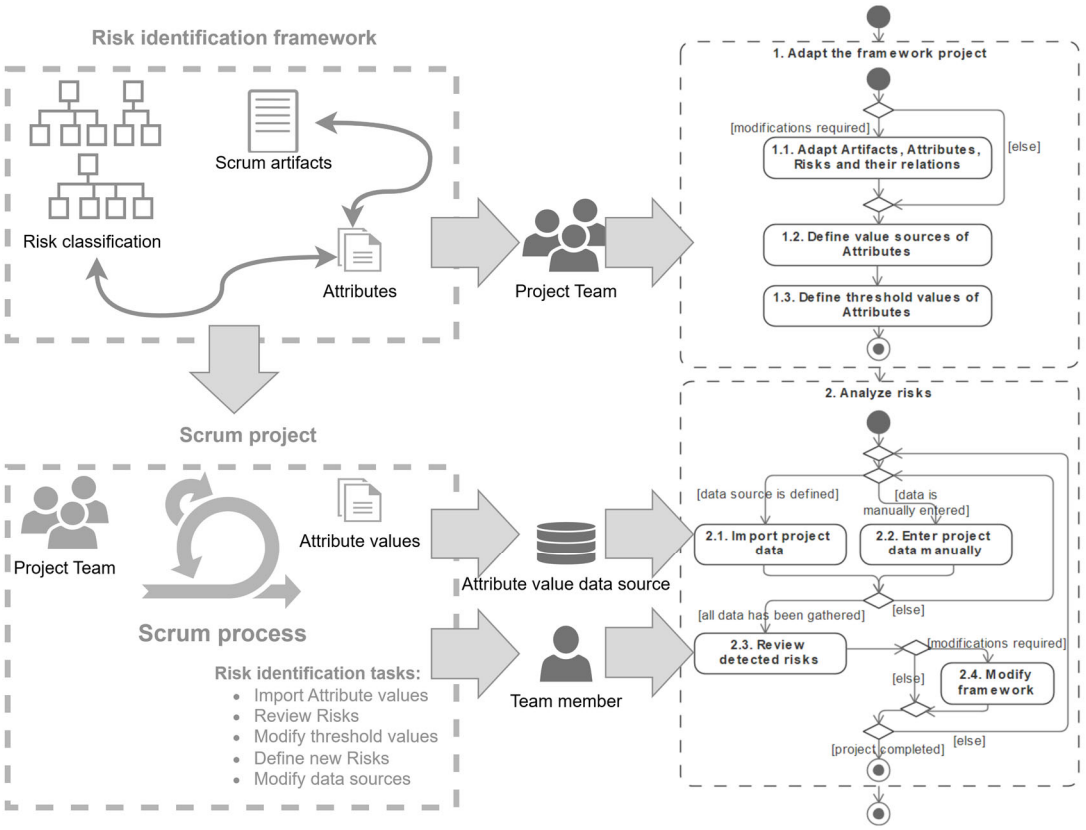

Figure 1. The main stages, steps, and participants in risk identification using the proposed framework.

The prepared framework, which encompasses a risk classifier, Scrum project artifacts, and attributes, is based on practical experience and analyzed research in the risk management area. The risk classification, lists of project artifacts and their attributes, and the relationships between attributes and risks are proposed as a basis for further improvement and adaptation of the project team according to the specifics of the project.

3.1. Structure of the Risk Identification Framework

The main concepts of the proposed Risk Identification Framework cover several important elements (Figure 2): risks and their classification into risk categories, Scrum artifacts and their attributes, and relationships between attributes and risks. Attributes can be associated with several different artifacts; therefore, an additional concept, artifact attribute, is required to store these associations, which also has threshold values for upper and lower threshold limits. Risks are associated with artifact attributes, and this information is stored in another additional concept, artifact attribute risk.

The attribute value concept stores project execution information gathered from data sources, such as the project execution environment or values entered manually by project team members. The concept of a registered risk is used to compile a set of risks that have

been detected when their associated attribute values surpassed a defined threshold and were subsequently identified as necessary through analysis by the project team.

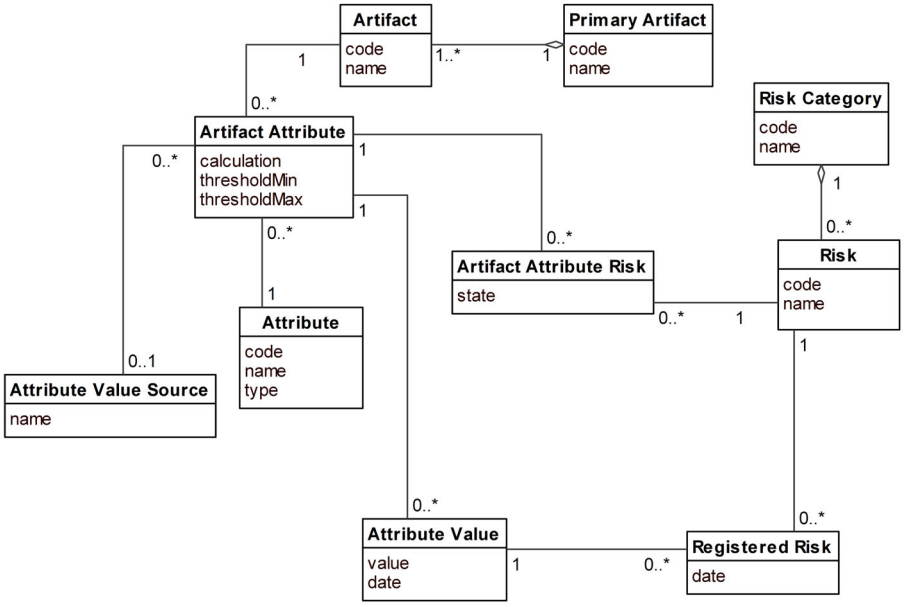

Figure 2. The conceptual model of the main elements of the Risk Identification Framework (0..*, 1..*—standard UML notation).

Table 2 describes and provides examples of the main elements of the Risk Identification Framework.

Table 2. Description of the main elements of the Risk Identification Framework.

Framework Element	Description	Element Attributes	Example Values
Primary artifact	Primary artifacts of the Scrum methodology	code; name	AR01; Product
Artifact	Specific artifacts decomposing primary artifacts	code; name	AR01.1; Product Backlog
Attribute	Information that needs to be tracked and stored in the Scrum project	code; name; type	AT005; Number of ready tasks; number
Artifact attribute	Attributes that need to be tracked for a given artifact	calculation; thresholdMin; thresholdMax	COUNTIF (Label, “Ready”); 20; -
Attribute value source	Data source of attribute values, usually a file, exported from Jira	name	DVS_Jira.xls
Risk category	Structural element, grouping the risks in the proposed risk classification	code; name	RC1; Requirements
Risk	Specific risk, decomposing a category according to the proposed classification	code; name	R001; Undefined requirements
Artifact attribute risk	Risk that is monitored for a specific couple of artifact and attribute	state	detected
Attribute value	Value imported from data source	value; date	9; 02/05/2024
Registered risk	Risk that was detected and registered	date	02/05/2024

The framework proposes basic risk classification by introducing nine risk categories for Scrum projects, which provide a structured approach to organizing the risks. Each risk category encompasses specific risks, as presented in Figure 3. The proposed risk classification is based on analyzed research [17–20] where similar naming and definitions of risk categories are used. The proposed categories and their risks serve as an initial reference for the project team. Using this classification as a foundation, the team is expected to develop a tailored list of risks specific to the project under development.

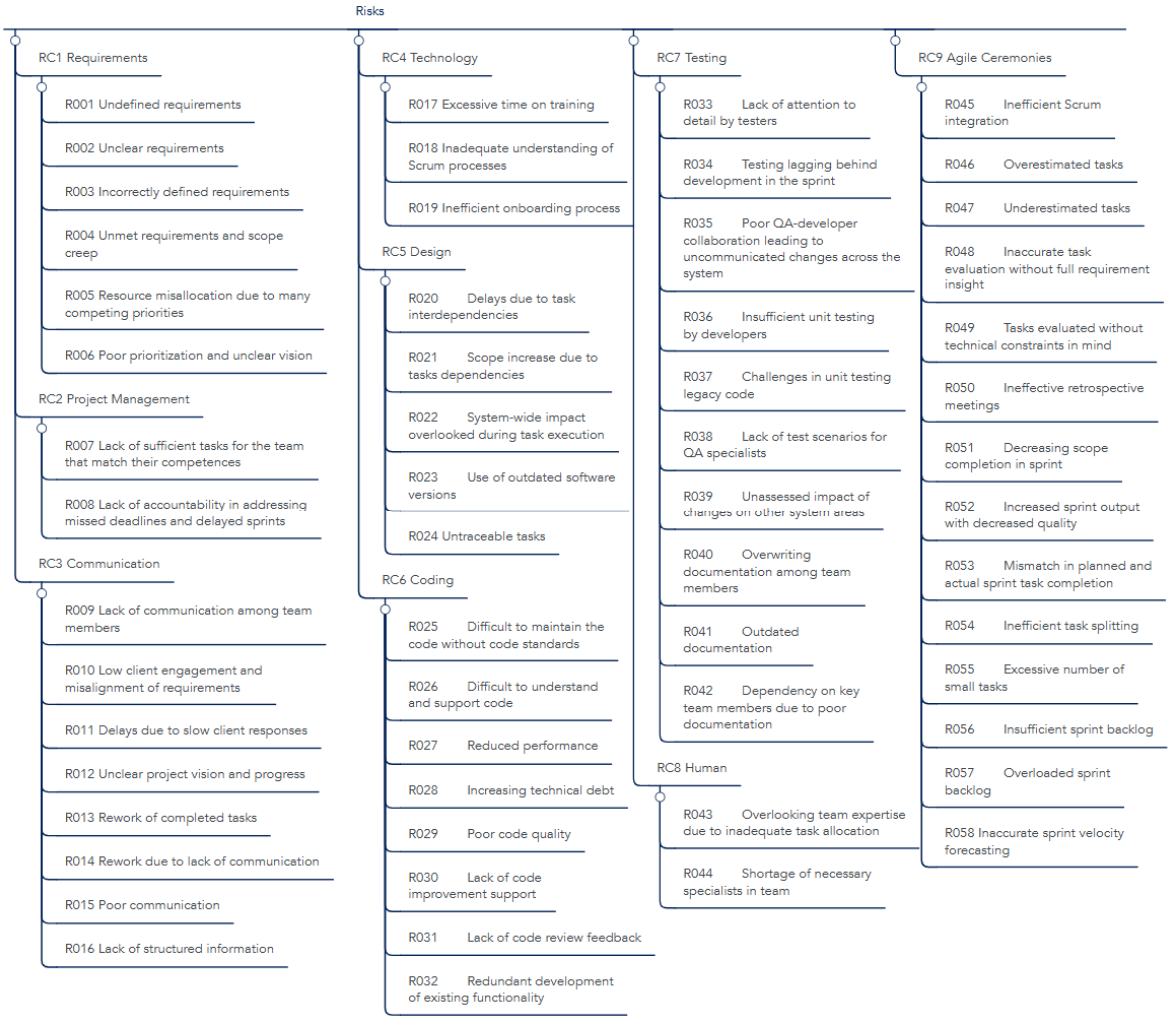

Figure 3. The proposed risk classification encompassing risks and their categories.

We propose a structured approach to identifying Scrum artifacts based on Scrum principles. The three main categories align with the primary artifacts of the Scrum methodology: Product, Sprint, and Increment (Figure 4). Each category is further detailed, with multiple specific artifacts identified under each. This proposed structure is a foundation for further improvement, enabling the project team to refine the recommended list of artifacts as needed.

Attributes denote the information that needs to be tracked and stored for a given artifact. The status of an artifact is monitored based on the values of its attributes, and deviations from the expected values indicate that the artifact has become risky. Since the same attribute can be associated with multiple artifacts, attributes can be identified separately and then linked to the respective artifacts. The attribute type determines how an attribute obtains its value:

- number—attribute is assigned a numeric value, either entered manually or imported from an external source (e.g., from Jira through an exported Excel spreadsheet).
- calculated—derived using other attribute values.

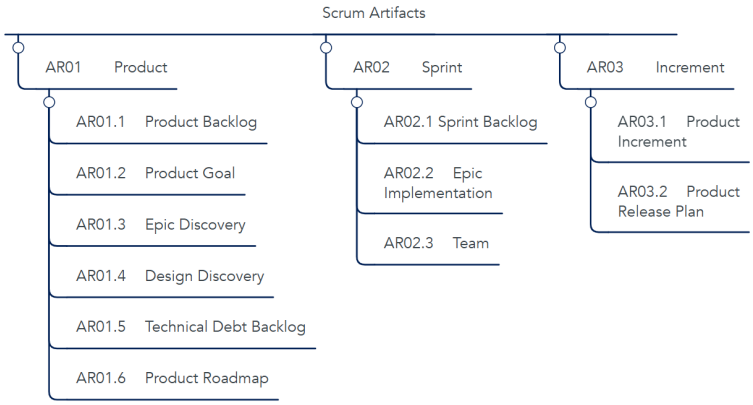

Figure 4. The proposed classification of artifacts of the Scrum project.

The full recommended set of attributes for each artifact is outlined in https://github.com/lincepo/riskIdentification (accessed on 3 January 2025), along with the suggested associations between attributes and risks across various risk categories. The fragment of the proposed attributes and associations with risks is presented in Table 3, which demonstrates attributes for the Product Backlog Artifact and their associations with the risks within the Requirements category. Using the proposed structure as the foundation, the project team can customize the associations between attributes and risks to suit the specific needs of their project.

Table 3. List of attributes recommended for Artifact AR01.01 Product Backlog with related risks (an 'x' indicates a relationship).

Artifact	Attributes		Risks					
			RC1 Requirements					
			R001 Undefined Requirements	R002 Unclear Requirements	R003 Incorrectly Defined Requirements	R004 Unmet Requirements and Scope Creep	R005 Resource Misallocation Due to Many Competing Priorities	R006 Poor Prioritization and Unclear Vision
AR01.1 Product Backlog	AT001	Number of errors		x	x	x		
	AT005	Number of ready tasks	x	x		x		x
	AT033	Average of tasks in epic	x					x
	AT035	Number of prepared sprints	x	x				x
	AT036	Percentage of high-priority tasks			x	x	x	x
	AT037	Percentage of low-priority tasks					x	x
	AT041	Number of tasks without epic		x	x			x
	AT042	Number of epics	x				x	x
	AT046	Number of initial tasks	x	x			x	x
	AT047	Number of technical tasks	x			x		

Figure 5 illustrates the distribution of associated attributes across different artifacts in the proposed structure. A greater number of attributes associated with an artifact allows for analyzing more diverse aspects of the artifact, which can be linked to risks. However, the hierarchy of artifacts should also be considered. For example, the first primary artifact (AR01 Product) has a larger number of sub-artifacts, each with fewer associated attributes, compared to the sub-artifacts of the second primary artifact (AR02 Sprint), which are fewer in number but have more associated attributes.

While the number of attributes assigned to artifacts reflects the level of detail in artifact analysis, it is equally important to consider the number of risks associated with each attribute and, consequently, their related artifacts. Figure 6 illustrates comparative statistics

for each risk category's associations with artifacts in the proposed framework structure. A higher number of risks associated with specific attributes (and their corresponding artifacts) enables the identification of more potential risks. However, not all identified risks need to be considered, as some may be less relevant to the specific project. To address this, the proposed framework includes an adaptation step in which the team selects the most significant risk and artifact associations based on team experience and knowledge about the project. Additionally, the proposed linking highlights specific areas in the project where risks are most likely to occur. For example, the largest number of associations is between the artifact AR02.1 Sprint Backlog and the risk category RC9 Agile Ceremonies. This suggests that this area represents a particularly risk-sensitive aspect of the project.

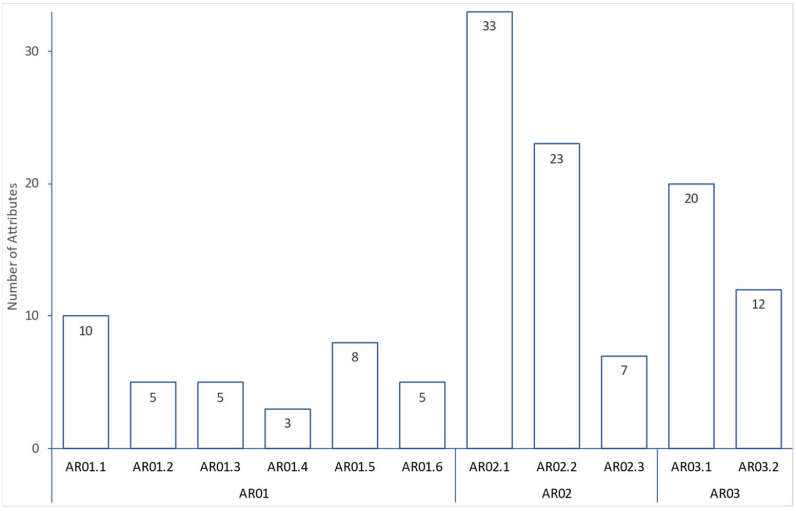

Figure 5. The number of attributes assigned to each artifact in the proposed framework.

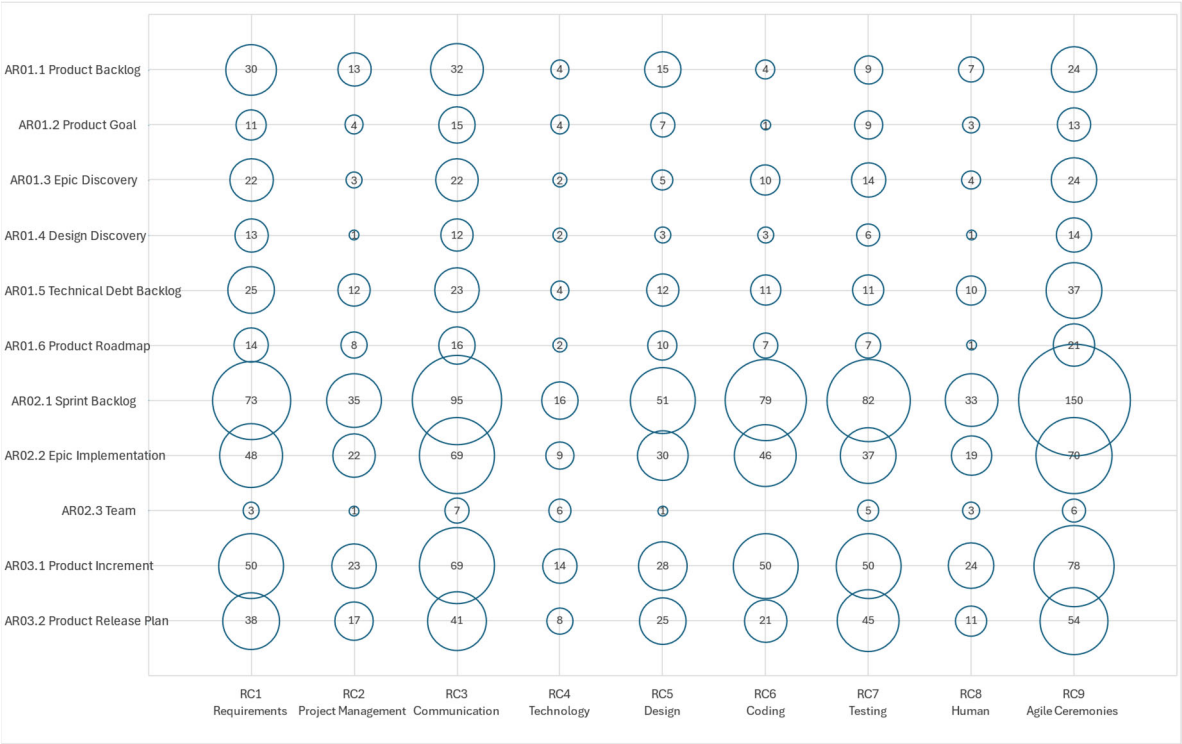

Figure 6. Statistics on the number of risks assigned to artifacts in the proposed framework.

The proposed framework application process involves two main stages: adapting the framework to the specific project and performing risk analysis. Subsequent sections examine these stages in more detail.

3.2. Adapting the Framework for Use in the Specific Project

The first stage of applying the framework is adapting the proposed lists of artifacts, attributes, and risks to suit the specific project (Figure 7). While the proposed framework structure can be used as-is, the team should assess whether it fully aligns with the specifics of their project. The team should start by selecting the relevant artifacts if adaptation is needed. In the proposed framework structure, attributes and their associated risks are already mapped to each artifact. However, the team has the flexibility to customize this structure by adding new artifacts, attributes, risk categories, or specific risks as needed. While customizing the artifacts, risks, and attributes for the project, the team must ensure that all selected artifacts have at least one attribute assigned (so that it would be possible to track the artifact status). The framework offers complete flexibility, allowing the team to create a structure and mappings that best reflect the project situation. From a practical standpoint, if teams within a company follow similar Scrum principles and use consistent tooling, they can adapt the framework once and reuse it across multiple projects, thus saving time and ensuring consistency.

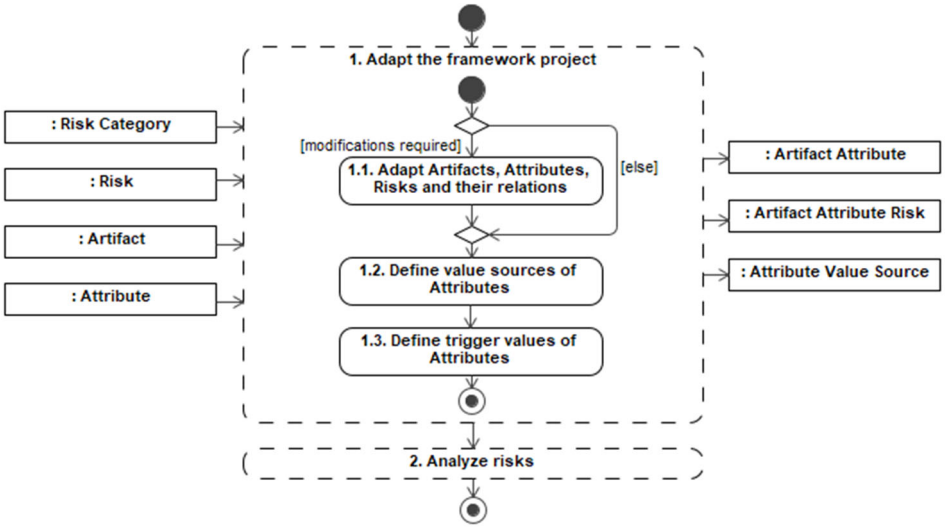

Figure 7. The stage of adapting the framework project.

The important step in the adaptation stage is defining external sources that provide data about the project's progress (Step 1.2 in Figure 7). The team needs to select a source file for each artifact and map the required attribute values and expressions from the chosen source. In Scrum projects, Jira is commonly used for project management, and .csv files exported from Jira can be the source of project execution information. The framework's attributes must then be mapped to specific cells in the exported spreadsheet. While the framework includes default mappings, the team can select their particular data sources and define their mappings. Additionally, the team must specify the time intervals for gathering data from sources, ensuring that the risk-related information is updated regularly. This step also enables the creation of derived attributes based on the values of other attributes. For example, the deviation of velocity can be calculated using the standard deviation formula applied to the last three velocity values gathered.

The adaptation stage's final step is defining threshold values to determine when an attribute is considered risky (Step 1.3 in Figure 7). Both minimum and maximum threshold values can be set if needed; however, at least one of these thresholds must be defined.

Once the initial framework adaptation is complete, the team moves on to the execution of the project. During this phase, they collect and analyze project execution data from the defined sources. This information is the foundation for identifying risks, as outlined in the next section.

3.3. Analyzing Risks

Once the adaptation is complete, the next stage is the risk analysis (Figure 8), which is conducted during the execution of the project. Project execution data are periodically imported from data sources and mapped to the corresponding attributes as attribute values. In most cases, data import occurs bi-weekly, aligning with the two-week sprint cycle typical in Scrum methodology. However, the team may adjust the frequency of data tracking as needed. It should be noted that reducing the frequency of data gathering is not advisable, as it can lead to losing visibility and control over the project. Attribute values can also be entered manually by team members. In such cases, ensuring that both the imported and manually entered values are updated simultaneously is important. Failure to do so may result in inconsistencies, particularly when derived attributes depend on other attribute values.

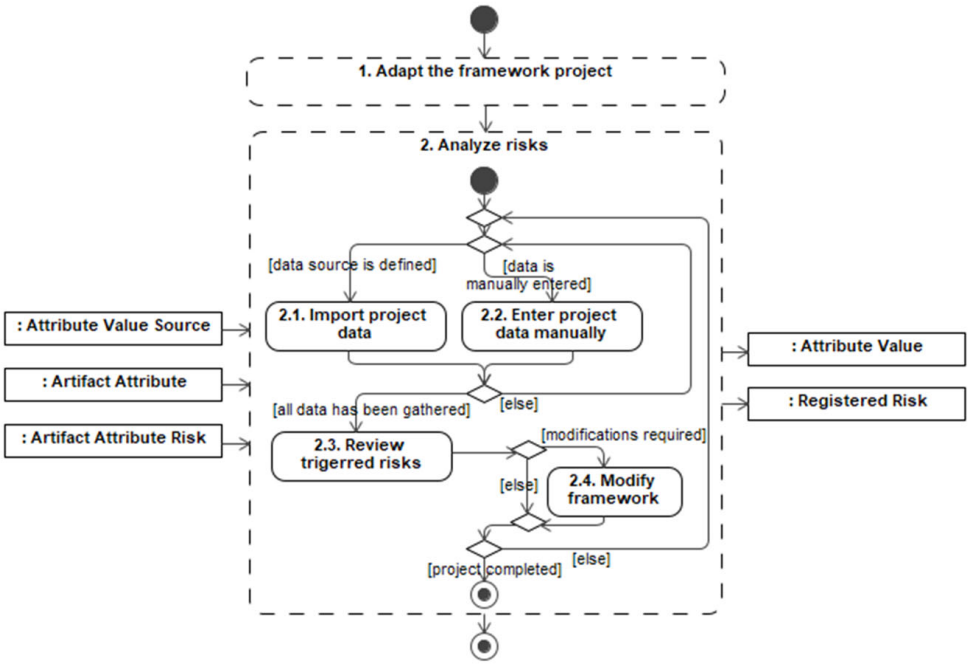

Figure 8. The stage of analyzing risks.

When data are gathered from the data source or entered manually, the next step in the process is reviewing risks associated with attributes that have reached their thresholds. These risks are presented to the team for review (Step 2.3 in Figure 8), where a decision is made on which attribute artifact risks should be registered for further analysis.

The states of the attribute artifact risk are illustrated in Figure 9, which demonstrates the progression of each risk through three states: Monitored (during the adaptation stage, when the team selects the risk for ongoing observation), Detected (when the gathered attribute value exceeds its threshold), and finally Registered or Rejected (based on the team's evaluation of the risk's significance). Detected risks are not automatically registered; instead, team members assess whether the artifact with the detected risk truly warrants concern.

For example, a significant drop in velocity might raise a flag. Still, if the decrease occurs during a holiday season, it may be an expected fluctuation rather than an actual problem.

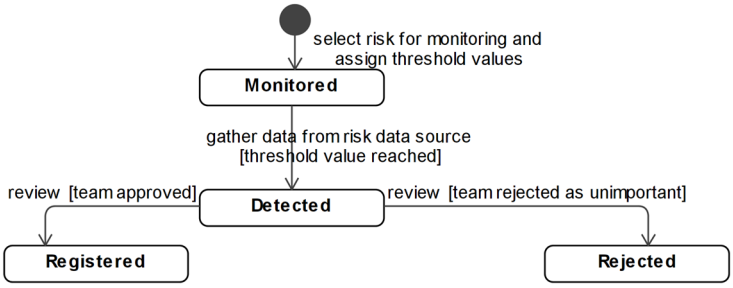

Figure 9. States of the artifact attribute risk class.

The risk analysis stage should be conducted continuously throughout the execution of the project. The framework should be adjusted accordingly if the team identifies any shortcomings in the adapted artifacts, attributes, risks, or their associations. Furthermore, new data sources may be added, thresholds may require fine-tuning, and new attributes may be identified as the project progresses. However, it is important to consider the impact of these modifications on the calculated attribute values, as these calculations may rely on previously imported data. Adjustments must be made carefully to maintain the integrity of the calculations. In general, the framework is designed to evolve during project execution. As the team's knowledge of the project and its environment deepens, the framework is refined to enable more accurate risk identification.

3.4. Implementation of the Proposed Risk Identification Methodology

The proposed methodology was implemented using a tool called RIS (Risk Identification System), which was designed to execute the main steps of the methodology. The RIS application provides users with a platform to detect and analyze risks throughout the project lifecycle. While using RIS in the adaptation stage, the user can adapt the framework to his project by choosing all necessary artifacts, assigning attributes, and linking risks with attributes. The user also defines value sources of attributes based on the .csv files exported from the Jira project management environment. Each attribute is assigned threshold values to enable risk detection. In the second stage, users regularly import .csv files exported from Jira. The tool processes the imported data, calculates attribute values for all artifacts, and identifies instances where attribute values reach predefined thresholds. A screenshot of the tool, showing the number of attributes that have met their thresholds, is presented in Figure 10. The user can see which artifacts have attributes reaching the threshold and thus detecting risks during the particular phase.

Also, the user can analyze attributes that reached the threshold (Figure 11). The tool highlights threshold-reaching attributes in red, allowing users to locate and analyze relevant information. If needed, users can also view the history of a specific attribute for deeper analysis. Additionally, the tool displays the potential risks associated with each attribute (Figure 11), allowing users to select and register relevant risks as part of the risk identification process.

The implemented RIS application was used during the experimental evaluation of the proposed framework, as detailed in the next section.

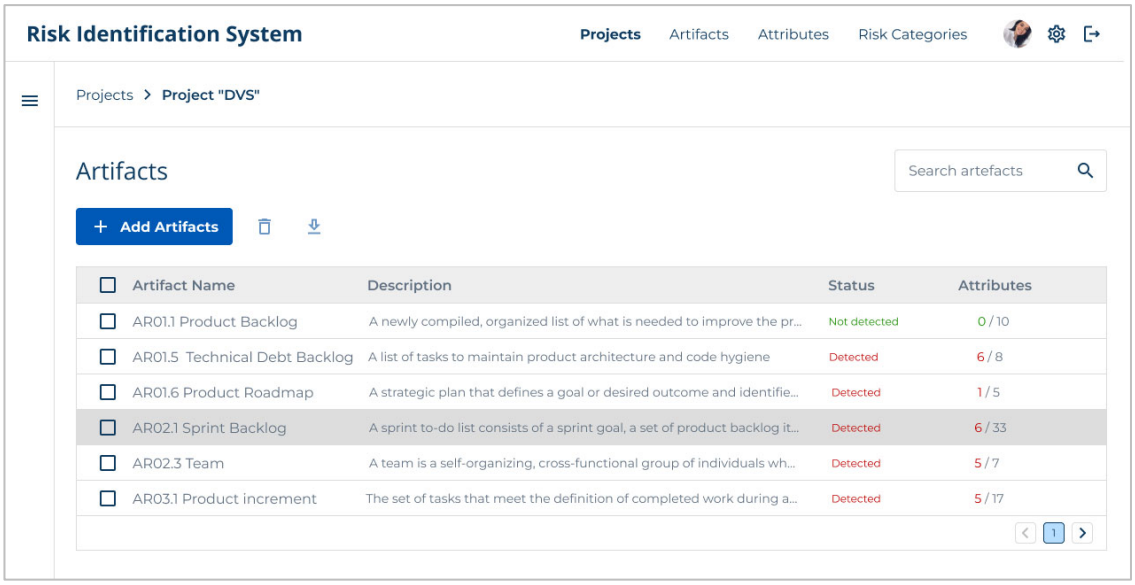

Figure 10. Risk identification framework management application: Artifacts and their statuses.

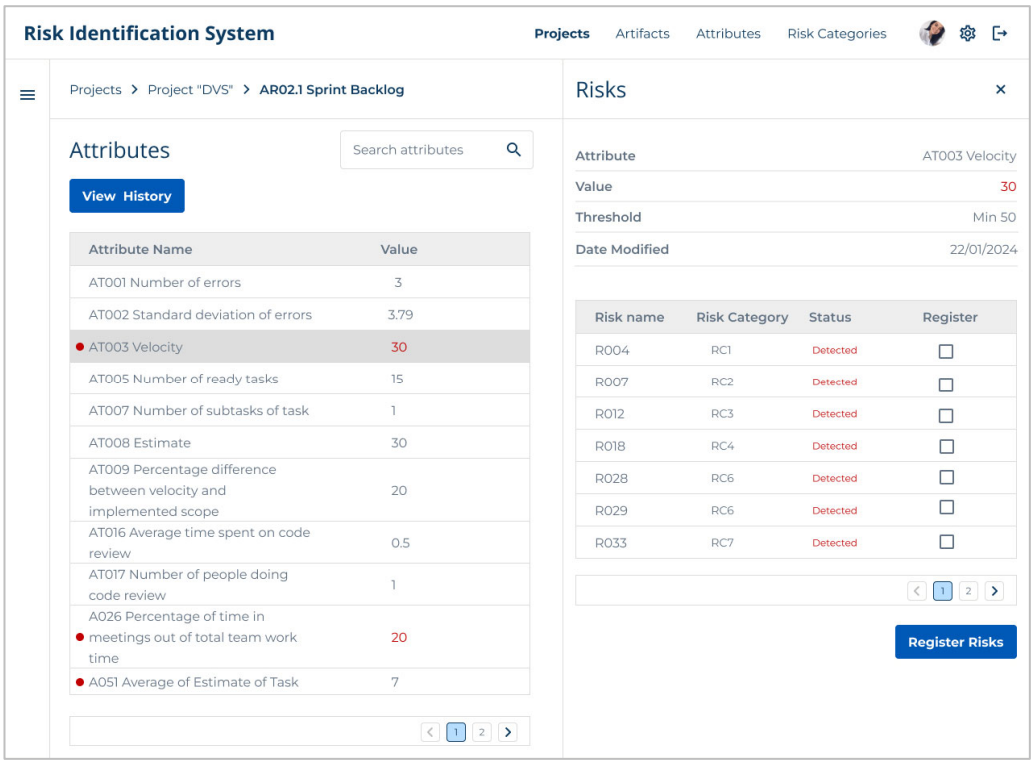

Figure 11. Risk identification framework management application: Attributes and related risks.

4. Experimental Evaluation of the Risk Identification Framework

4.1. Experiment Setup

To evaluate the proposed Risk Identification Framework, an experimental study was conducted encompassing three projects in which the framework was introduced in different phases of the project lifecycle. Despite differences in project phases, all projects shared common characteristics, including team sizes ranging from 12 to 20 members, a uniform project duration of 12 months, and a consistent bi-weekly data upload schedule:

- Project 1—development of the employee assessment and the candidate selection and evaluation platform. The platform had to support individual tests and surveys completed by the assigned users. The results of the tests were to be used to generate reports, dashboards, and advice for personal and professional development. Client: Employment Agency. Development team size: 20 people. Time period: 12 months.
- Project 2—development of an e-banking and mobile banking app. The online platform had to allow users to track their balance, open additional accounts, and make local and international money transfers conveniently and securely. Additionally, the platform was required to have complementary features such as money requesting and financial insights. Client: Bank. Development team size: 12 people. Time period: 12 months.
- Project 3—development of an employee salary calculation and personnel management system, which was required to ensure the company's personnel appointments and personal data management, monitoring and controlling working hours, and wage management. Client: Insurance company. Team size: 16 people. Time period: 12 months.

In each project, the Scrum master and the project team selected the artifacts to monitor, tailoring their choices to the specific requirements and context of the project. Each project adhered to the proposed framework methodology, implementing all necessary steps to ensure consistency.

4.2. Results of the Framework Adaptation Stage in Experimental Projects

All three projects underwent the framework adaptation stage, where the project teams were provided with the proposed structure of artifacts, attributes, and risks. Teams performed the refinement of the elements of the structure, their associations and threshold values based on the expertise of the team. Table 4 summarizes the adaptation results, illustrating the utilization of artifacts, attributes, and risks for each project, compared to the proposed initial structure.

Table 4. Results of the adaptation stage in experimental projects.

Quantitative Indicator	Project 1	Project 2	Project 3
Artifacts used	55%	73%	82%
Attributes used	89%	93%	70%
Risk categories used	78%	78%	89%
Risks used	69%	83%	86%
Risk-attribute associations used	48%	82%	87%

All projects used Jira to export project execution data to the Excel file and mapped the data from Excel to corresponding attribute values in the framework application.

Project 1 was experimented upon during the initiation and execution phases over one year. Consequently, the team opted to focus on artifacts, attributes, and risks relevant to the initial stages of the project. This decision led to the exclusion of several artifacts, including AR01.2 (Product Goal), AR01.3 (Epic Discovery), AR01.4 (Design Discovery), AR01.6 (Product Roadmap), AR02.2 (Epic Implementation), and AR03.2 (Product Release Plan). The team determined that the characteristics of these artifacts could be effectively represented through attributes associated with other related artifacts in their project. For instance, attributes such as AT042 (Number of Epics) and AT046 (Number of Initial Tasks), which are tracked within AR01.1 (Product Backlog), were deemed sufficient in this case for identifying risks such as R006 (Poor prioritization and unclear vision) and R015 (Poor communication). Despite a selective focus on artifacts, the adoption rate of risk categories was relatively high, as seven out of the nine risk categories were utilized. Excluded categories were RC5

(Design), RC8 (Human), and several risks from categories RC6 (Coding) and RC7 (Testing), based on the same rationale as the exclusion of artifacts.

Project 2 and Project 3 were introduced with the application of the proposed framework during the later phases of their lifecycles. At the time of the framework application, both projects were mature, characterized by continuous delivery and well-established operational practices. Due to the absence of anticipated specific risks, these projects have opted to monitor a broader set of artifacts than those of Project 1. The adoption of risk categories was also significantly higher compared to Project 1, with Project 2 and Project 3 showing adoption rates of 78% and 89%, respectively. On the other hand, Project 3 adopted the smallest number of attributes due to the decision to exclude unsupported attributes—those for which data were not gathered within their project management environment, e.g., AT041 (Number of tasks without epic), AT031 (Number of open versions). Although no specific risks were anticipated, the teams aimed to monitor a diverse array of potential risks to ensure that projects remain on schedule, as reflected in the high percentage of adopted risk-attribute associations.

4.3. Results of the Risk Analysis Stage in Experimental Projects

A summary of the risk detection and registration statistics observed during the risk analysis stage is presented in Figure 12. The number of detected risks for each project is calculated by summing the unique risks identified per sprint for each artifact. The variation in the number of attributes that reached the threshold was influenced by several factors, including the project lifecycle stage during which the framework was applied, the team's expertise in selecting artifacts, attributes, and risks for monitoring, and the choice of threshold values for attributes.

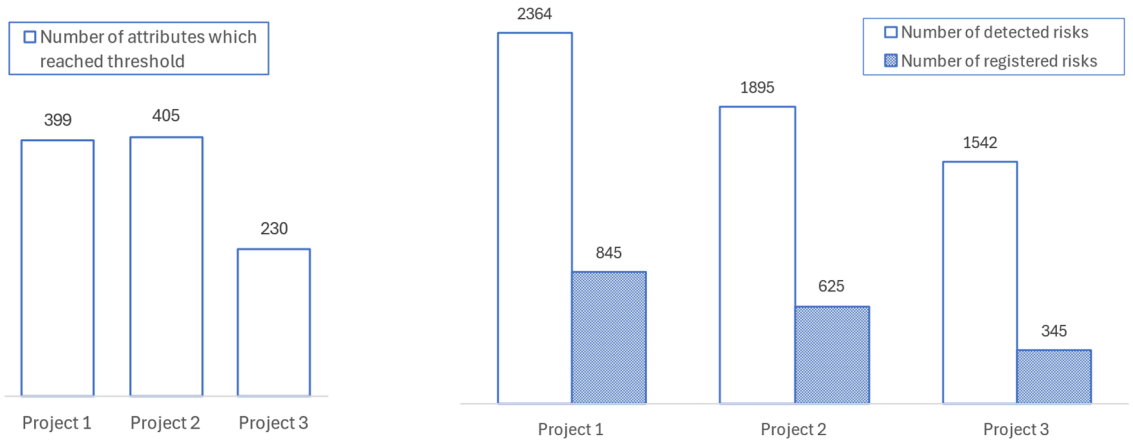

Figure 12. Results of the risk analysis stage in experimental projects: (**a**) Number of attributes that triggered risk detection, because they reached the upper or lower limit of their threshold value; (**b**) Numbers of detected risks compared to registered risks, which were deemed significant by the project team.

In Project 1, the team opted to track fewer artifacts, yet a significant proportion of their attributes reached the threshold in each sprint. As a result, 399 occurrences of attributes reaching the threshold were identified, leading to the detection of 2364 potential risks per Sprint per artifact, corresponding to an average of 118 risks per sprint. While the team considered increasing the threshold values, they ultimately decided to adhere to the predefined thresholds as a guideline. Despite the substantial number of potential risks identified, the team successfully filtered and registered only those risks that related to

attributes with the largest deviations from the thresholds. In total, 36% of the detected risks were registered in Project 1.

In Project 2, a larger number of artifacts and their attributes were tracked, which increased the likelihood of occurrences reaching the threshold. However, this number of occurrences was also influenced by the team's choice of threshold values, as a higher tolerance for failure resulted in fewer attributes exceeding the threshold. Compared to Project 1, Project 2 emphasized the risk categories RC6 (Coding) and RC7 (Testing). Ultimately, 33% of the detected risks were registered.

In Project 3, the framework was introduced mid-project. To minimize the probability of deviations in attributes, the team selected a smaller subset of attributes to track. This approach, combined with effective collaboration within the team and with the client, resulted in minimal deviations in attribute values and a lower number of potential risks. As in Project 1, the team registered only those detected risks that were associated with the largest deviations from the thresholds in attribute values.

The following section provides a more detailed analysis of the framework application's results. Project 1 was selected for an in-depth examination of the risk analysis process, focusing on statistics from the perspectives of artifacts, risks, and their categories.

4.4. Detailed Risks Analysis Stage Results for Project 1

Figure 13 illustrates the statistics of threshold-reaching occurrences for attributes across sprints in Project 1. During the adaptation stage, the Project 1 team selected six artifacts for monitoring: Product Backlog, Technical Debt Backlog, Product Roadmap, Sprint Backlog, Team, and Product Increment. The color-coding in Figure 12 corresponds to these artifacts, enabling a comparative analysis of threshold-reaching occurrences for attributes associated with each artifact during the sprints.

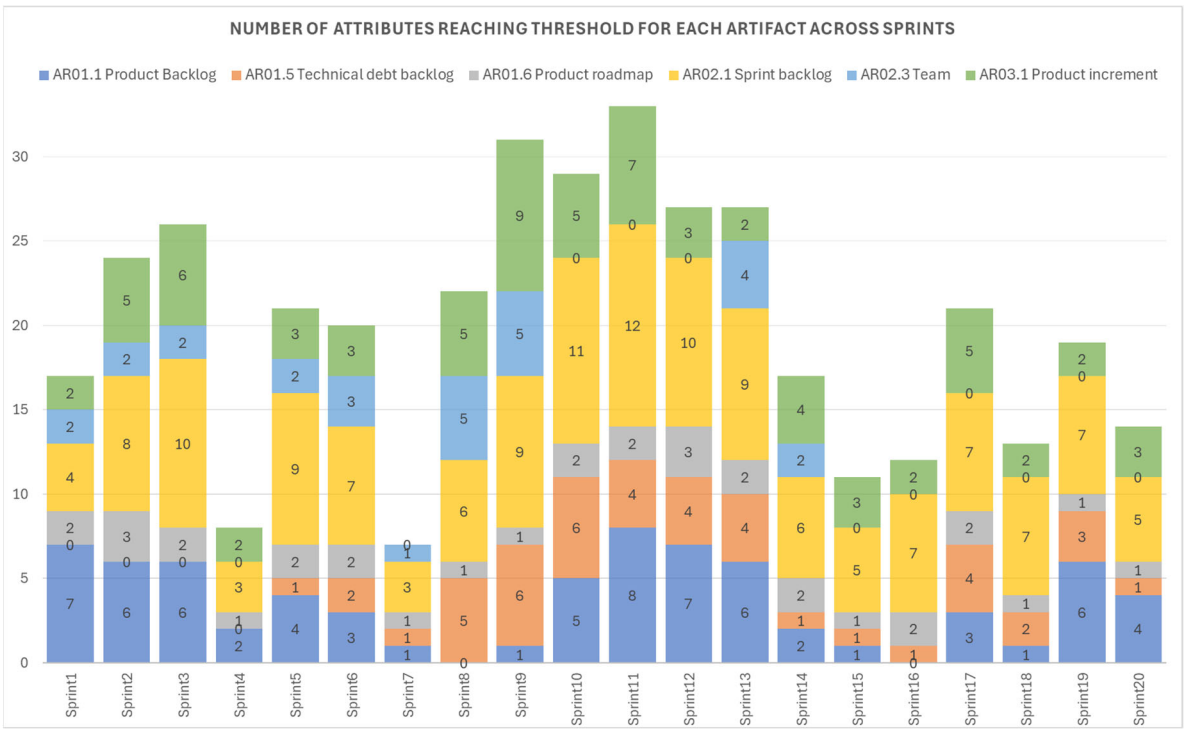

Figure 13. Statistics on the occurrences of attributes reaching the threshold for each artifact by sprint.

Although the number of attributes surpassing the threshold varies across sprints and artifacts, several trends are apparent. During the initial third of the project, Sprint 3 demonstrates the highest peak, with an attribute that reaches the threshold count of 26. In

the middle third of the project, the highest peaks are observed in Sprints 9 and 11, with attribute counts of 31 and 33, respectively. In the final third of the project, a smaller peak is observed in Sprint 17, with an attribute count of 21. These peaks align with the most challenging phases of the project, during which the team needed to address multiple focal areas, including RC1 (Customer Collaboration), RC3 (Communication), and RC9 (Agile Practices). For example, in Sprint 3, the team faced significant communication challenges (RC3) and difficulties adhering to Agile principles (RC9). Similarly, preparation for new functionality in Sprint 11 resulted in challenges related to customer collaboration (RC1 and RC3), issues with code quality control (RC6), and disagreements concerning Agile practice implementation (RC9).

Across all 20 sprints in Project 1, the Primary Artifact AR01 Backlog (encompassing AR01.1, AR01.5, and AR01.6 artifacts) was associated with 153 occurrences of its related attributes reaching the threshold. For the Artifact AR01.1 Product Backlog, the key sprints identified for risk activity are Sprints 1, 11, and 12, with risks being most prominent during the mid-phase of the project timeline. In particular, there was a peak of seven threshold-reaching occurrences in Sprint 1 and Sprint 12 and a peak of eight occurrences in Sprint 11, indicating increased challenges associated with the Product Backlog during these periods. The team attributed the peak in Sprint 11 to new client-requested functionality, while the spike in Sprint 1 was dismissed due to the instability of processes at the project's start. The other artifact, AR01.5 Technical Debt Backlog, had 46 occurrences of attributes reaching thresholds that invoked risk detection. These occurrences tended to concentrate later in the project, starting from Sprint 5 and reaching six occurrences in some periods. This suggests that addressing technical debt has gotten more complex and was given more focus in mid-stages when the team was not focusing on the new functionality. The Artifact AR01.6 Product Roadmap has the occurrences of attributes reaching the threshold fluctuating between 1 and 3, indicating an ongoing challenge with planning and roadmap development. Risky occurrences in practically every sprint suggest continuous attention to Product Roadmap challenges throughout the project.

The other primary artifact AR02 sprint (encompassing AR02.1 and AR02.3 artifacts) had a total of 173 occurrences reaching the threshold, thus triggering risk detection. Compared to AR01, from the point of view of AR02, Sprint 1 was better prepared, resulting in fewer attributes surpassing the threshold. However, Sprint 11, which involved the introduction of new functionality, revealed a misalignment within the team, and the preparation for this sprint was not optimal. The later sprints, with only 5–7 occurrences of triggering risks, demonstrate improved stability in Sprint Backlog management.

The Primary Artifact AR03 Increment (encompassing AR03.1 Artifact) had 52 occurrences exceeding the threshold. Sprints 3, 9, and 11 were particularly notable, with 6, 9, and 7 attribute counts, respectively. These peaks were associated with the deployment of critical functionalities within the project. During these sprints, the team was required to concentrate on finalizing and deploying existing functionalities while simultaneously beginning the analysis of upcoming features. This shift in focus increased the potential for emerging risks due to distractions and competing priorities.

The distribution of detected risks by artifact is illustrated in Figure 14. The number of detected risks represents the total unique risks identified per sprint, aggregated across each artifact. Although the number of detected risks is high, the proportion of registered risks compared to detected risks is relatively low, ranging between 10% and 51%. In Project 1, the team initially did not understand the risk management process. When the framework and associated process for risk analysis were introduced, the team initially intended to use it solely to identify risks without formally registering them. They assumed risks were adequately addressed during the sprint cycle and dismissed formal documentation as

"waterfall bureaucracy". However, after adopting the framework, the team discovered that the registration process required minimal effort and decided to register the most critical risks according to the proposed process. This change allowed the team to log risks and monitor their resolution over time systematically.

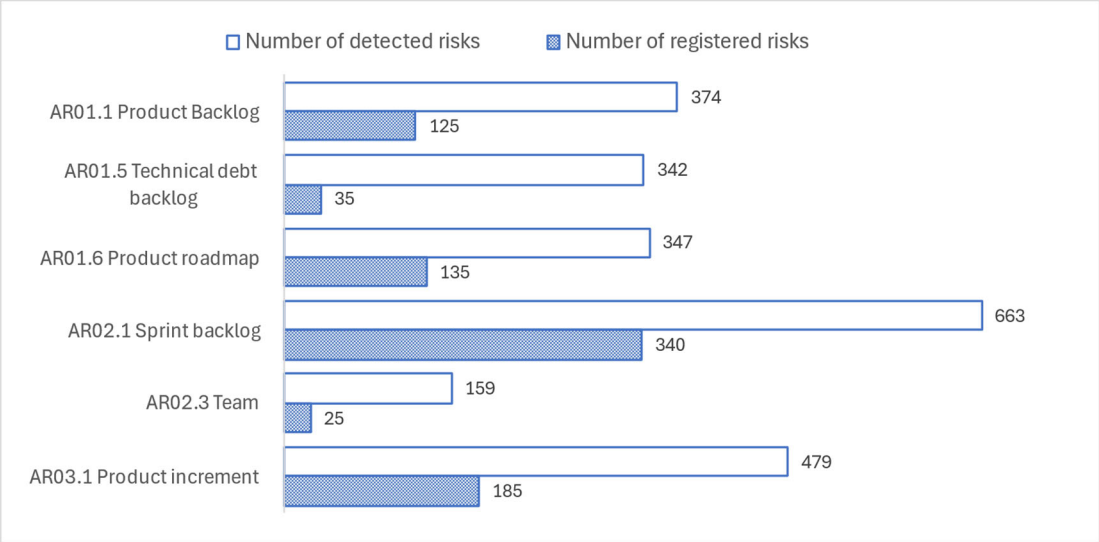

Figure 14. Comparison of detected and registered risks by artifact.

To demonstrate the distribution of detected risks across various sprints and risk categories, a heatmap is presented in Figure 15, where darker shades indicate a higher number of detected risks. This graphical representation facilitates the identification of sprints and risk categories with increased risk activity (risk categories RC5 and RC8 show no activity because they were excluded from the monitored risks during the adaptation stage). Notable peaks were observed in the risk categories RC3 (Communication) and RC9 (Agile Ceremonies), particularly during Sprints 3, 9, and 11. These peaks can be attributed to challenges in establishing an effective collaboration framework with the client in the project. The client inconsistently articulated their requirements and failed to provide all necessary details, leading to gaps in domain understanding and project execution. Additionally, discrepancies in the competencies of the project team and the clients' Agile practitioners further complicated the adaptation of Scrum practices. As a result, heightened focus was required to mitigate these risks.

It is important to note that risks across different categories are interconnected; emerging risks in one category can potentially impact others. This interconnectedness often arises when the team concentrates heavily on one domain, inadvertently neglecting others. Monitoring detected risks over time provides insights into whether these challenges are isolated incidents or indicative of persistent, systemic issues. For example, risks associated with RC9 (Agile Practices) were particularly prevalent from Sprint 8 to Sprint 13 in Project 1, suggesting potential chronic difficulties in this area.

In Project 1, the introduced framework facilitated an expanded risk analysis scope, encompassing the level of artifacts and their associated attributes, and the context of identified risks. As this approach was newly adopted, risk analysis was conducted during stand-up meetings, with the technical manager responsible for formally recording identified risks. The team consistently reviewed the status of registered risks and observed a shift in focus across different risk categories at various project stages. For example, risk R001 (Undefined Requirements) of the RC1 category was more frequently registered during the middle phase of the project. However, by the project's conclusion, although R001 continued

to be identified, the team prioritized risks from other categories, such as RC6 (Coding) and RC7 (Testing). In general, the team provided a positive evaluation of the framework introduced, noting improvements in risk identification, collaborative discussions, and systematic risk tracking processes. Tracking attributes using existing Jira data streamlined the workflow and increased the team's confidence in identifying critical risks in the project.

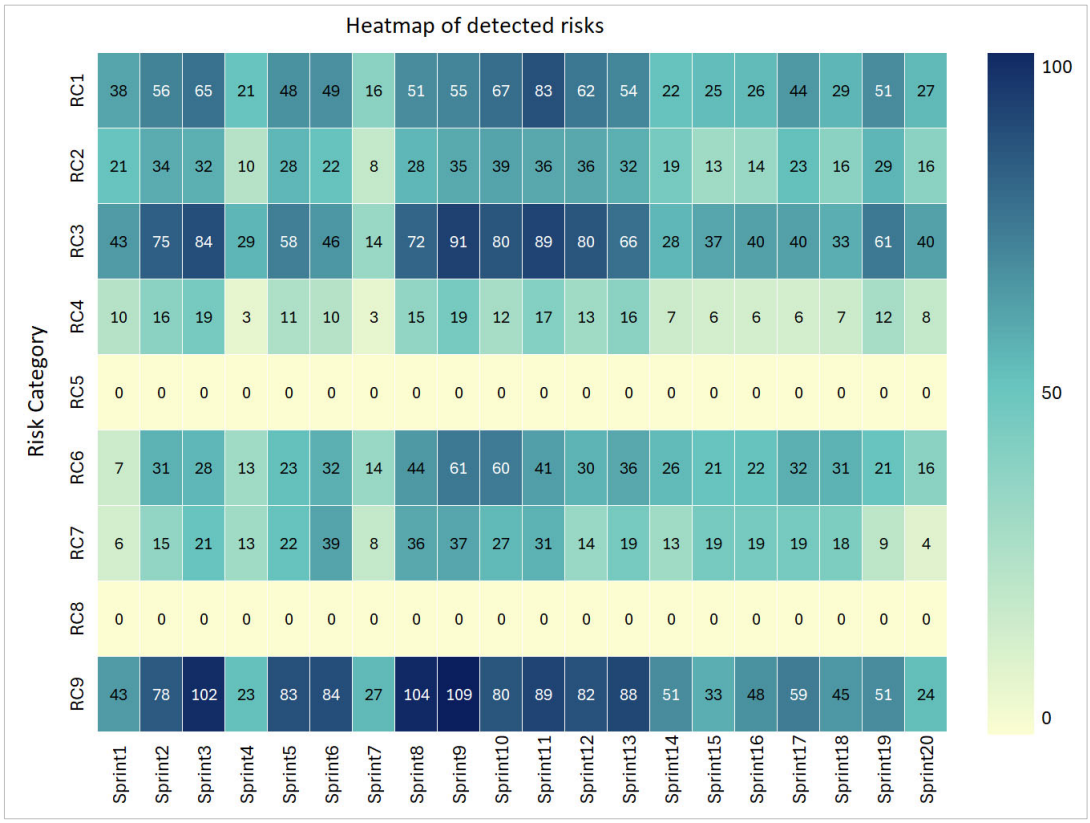

Figure 15. Heatmap of detected risks by sprint.

5. Discussion and Conclusions

A key strength of the proposed framework is its adaptability. Agile teams employ various approaches to risk identification, including the selection of tracked project data, thresholds, and risk registration principles. The framework accommodates this variability, allowing teams to tailor its structure and components to their specific context. This flexibility is essential, as no two Scrum projects are identical, and a one-size-fits-all approach to risk identification is insufficient. Furthermore, the framework enables teams to identify and document risky deviations with minimal effort by automating data collection from project management tools such as Jira. The proposed structure, which links project artifacts and attributes with risk classifications, allows for the real-time assessment of project status. It helps to identify potential risks by applying threshold values defined by the team. By leveraging these tools for data gathering and analysis, teams can continuously monitor the project situation and detect emerging risks immediately, preventing risks from escalating.

It is advisable for the team and its manager to implement the framework at an early stage of the project. This facilitates data collection and subsequent analysis. The team should focus on the key factors influencing the selection of the appropriate framework configuration. Initially, it is recommended to include all artifacts, emphasizing comprehensible and pertinent attributes to the team. The same approach applies to risk management. If the team lacks a clear understanding of the risks, their identification may lead to confusion and dissatisfaction. The list of attributes and risks can be expanded as the project progresses,

as the primary elements will have been stabilized, enabling broader project monitoring. However, reducing the number of attributes or risks is not recommended, as long-term mutual correlations may uncover latent risks. For complex projects with multiple teams, it is recommended that each team maintains its own set of attributes and monitored risks. Aggregating these elements across teams may introduce challenges in tracing root causes and complicate the development of risk mitigation strategies.

However, the framework has limitations. It relies on accurate and timely data imports from tools such as Jira. Delayed or inconsistent imports can result in incomplete risk assessments and missed critical risks. Moreover, the effectiveness of the framework depends on the team's understanding of the principles of risk identification and their active participation in the process. Furthermore, the reliance on threshold values for risk detection presents challenges, as improperly calibrated thresholds may generate false positives or fail to identify significant risks.

Experimental evaluations of the framework in several projects demonstrated its potential to improve team-wide risk awareness throughout the project lifecycle. For the teams, their most unexpected and important realization was an underestimation of the impact of long-term risks. The gathered historical project execution data facilitated the identification of persistent issues that, while appearing negligible within the scope of a single sprint, revealed their influence over extended periods. Including all team members in the risk analysis process eliminated the practice of confining risk evaluation and decision-making to senior members, fostering a collaborative approach to risk identification.

Future research will focus on improving the synchronization with the Jira project management tool. This will involve enabling the capability to flag risks directly within a project execution environment and convert them into actionable tasks. Additionally, efforts will be directed toward expanding the range of calculated attributes to encompass a broader project ecosystem. Another critical area identified during the experiments is the interdependence between attributes and risks. Specific attributes were observed to have dependencies, and similarly, the occurrence of one risk may cascade into the emergence of other related risks. This aspect will be further explored to develop methodologies for capturing and analyzing these interdependencies more effectively.

Author Contributions: Conceptualization, L.B. and L.Č.; software, A.N.; validation, L.B., A.N. and G.V.; data curation, L.B. and G.V.; writing—original draft preparation, L.B. and L.Č.; writing—review and editing, L.Č., L.B. and G.V.; visualization, L.B. and L.Č. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article, further inquiries can be directed to the corresponding authors.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Boehm, B. Software Risk Management. In *Proceedings of the 2nd European Software Engineering Conference, University of Warwick, Coventry, UK, 11–15 September 1989. Proceedings*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 1989; pp. 1–19. [CrossRef]
2. Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)—Seventh Edition and the Standard for Project Management*; Project Management Institute: Newtown Square, PA, USA, 2021; ISBN 9781628257120.

3. Tavares, B.G.; da Silva, C.E.S.; de Souza, A.D. Risk Management Analysis in Scrum Software Projects. *Int. Trans. Oper. Res.* **2017**, *26*, 1884–1905. [CrossRef]
4. Schwaber, K.; Sutherland, J. The Scrum Guide the Definitive Guide to Scrum: The Rules of the Game. Available online: https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf (accessed on 11 December 2024).
5. Bakker, K.D.; Boonstra, A.; Wortmann, H. The Communicative Effect of Risk Identification on Project Success. *Int. J. Proj. Organ. Manag.* **2014**, *6*, 138. [CrossRef]
6. Kuhrmann, M.; Fernandez, D.; Grober, M. Towards Artifact Models as Process Interfaces in Distributed Software Projects. In Proceedings of the IEEE 8th International Conference on Global Software Engineering, Bari, Italy, 26–29 August 2013.
7. Mihelič, A.; Hovelja, T.; Vrhovec, S. Identifying Key Activities, Artifacts and Roles in Agile Engineering of Secure Software with Hierarchical Clustering. *Appl. Sci.* **2023**, *13*, 4563. [CrossRef]
8. Wagenaar, G.; Overbeek, S.; Lucassen, G.; Brinkkemper, S.; Schneider, K. Working Software over Comprehensive Documentation—Rationales of Agile Teams for Artefacts Usage. *J. Softw. Eng. Res. Dev.* **2018**, *6*, 7. [CrossRef]
9. Wagenaar, G.; Helms, R.; Damian, D.; Brinkkemper, S. Artefacts in Agile Software Development. In Proceedings of the Product-Focused Software Process Improvement (PROFES 2015), Bolzano, Italy, 2–4 December 2015; pp. 133–148.
10. Almeida, F.; Carneiro, P. Perceived Importance of Metrics for Agile Scrum Environments. *Information* **2023**, *14*, 327. [CrossRef]
11. Ghane, K. Quantitative Planning and Risk Management of Agile Software Development. In Proceedings of the IEEE Technology & Engineering Management Conference (TEMSCON), Santa Clara, CA, USA, 8–10 June 2017; pp. 109–112.
12. Atlassian Jira Software. Available online: https://www.atlassian.com/software/jira (accessed on 11 December 2024).
13. Hammad, M.; Inayat, I.; Zahid, M. Risk Management in Agile Software Development: A Survey. In Proceedings of the International Conference on Frontiers of Information Technology (FIT), Islamabad, Pakistan, 16–18 December 2019; pp. 162–1624.
14. Garcia, F.; Hauck, J.; Hahn, F. Managing Risks in Agile Methods: A Systematic Literature Mapping. In Proceedings of the International Conference on Software Engineering and Knowledge Engineering (SEKE 2022), Pittsburgh, PA, USA, 1–10 July 2022.
15. Anes, V.; Abreu, A.; Santos, R. A New Risk Assessment Approach for Agile Projects. In Proceedings of the International Young Engineers Forum (YEF-ECE), Costa da Caparica, Portugal, 1–3 July 2020; pp. 67–72.
16. Ņikiforova, O.; Babris, K.; Kristapsons, J. Survey on Risk Classification in Agile Software Development Projects in Latvia. *Appl. Comput. Syst.* **2020**, *25*, 105–116. [CrossRef]
17. Khanna, E.; Popli, R.; Chauhan, N. Identification and Classification of Risk Factors in Distributed Agile Software Development. *J. Web Eng.* **2022**, *21*, 1831–1851. [CrossRef]
18. Esteki, M.; Javdani Gandomani, T.; Khosravi Farsani, H. A Risk Management Framework for Distributed Scrum Using PRINCE2 Methodology. *Bull. Electr. Eng. Inform.* **2020**, *9*, 1299–1310. [CrossRef]
19. Shrivastava, S.V.; Rathod, U. A Risk Management Framework for Distributed Agile Projects. *Inf. Softw. Technol.* **2017**, *85*, 1–15. [CrossRef]
20. Kumar, C.; Yadav, D.K. A Probabilistic Software Risk Assessment and Estimation Model for Software Projects. *Procedia Comput. Sci.* **2015**, *54*, 353–361. [CrossRef]
21. Adel, R.; Harb, H.; Elshenawy, A. A Multi-Agent Reinforcement Learning Risk Management Model for Distributed Agile Software Projects. In Proceedings of the Tenth International Conference on Intelligent Computing and Information Systems (ICICIS), Cairo, Egypt, 5–7 December 2021.
22. Singh, S.; Madaan, G.; Singh, A.; Sood, K.; Grima, S.; Rupeika-Apoga, R. The AGP Model for Risk Management in Agile I.T. Projects. *J. Risk Financ. Manag.* **2023**, *16*, 129. [CrossRef]
23. Lunesu, M.I.; Tonelli, R.; Marchesi, L.; Marchesi, M. Assessing the Risk of Software Development in Agile Methodologies Using Simulation. *IEEE Access* **2021**, *9*, 134240–134258. [CrossRef]
24. Tavares, B.G.; Keil, M.; Sanches da Silva, C.E.; de Souza, A.D. A Risk Management Tool for Agile Software Development. *J. Comput. Inf. Syst.* **2020**, *61*, 561–570. [CrossRef]
25. Chaouch, S.; Mejri, A.; Ghannouchi, S.A. A Framework for Risk Management in Scrum Development Process. *Procedia Comput. Sci.* **2019**, *164*, 187–192. [CrossRef]
26. Lopes, S.; Gratão de Souza, R.; Contessoto, A.; Luiz de Oliveira, A.; Braga, R. A Risk Management Framework for Scrum Projects. In Proceedings of the 23rd International Conference on Enterprise Information Systems, ICEIS 2021, Online Streaming, 26–28 April 2021; Volume 2.
27. de Souza Lopes, S.; de Souza, R.C.G.; de Godoi Contessoto, A.; de Oliveira, A.L.; Braga, R.T.V. Automated Support for Risk Management in Scrum Agile Projects. In *Enterprise Information Systems*; Springer: Cham, Switzerland, 2022; pp. 236–255. [CrossRef]
28. Marzuki, P.; Munawar, M.; Budi, T.; Puteri, S.; Akbar, H.; Firmansyah, G. Criticism of the Risk Management Process in Scrum Methodology. In Proceedings of the International Conference on Electrical and Information Technology (IEIT), Malang, Indonesia, 15–16 September 2022.

29. Devesh Kumar, S.; Makhija, R.; Batta, A. Software Vulnerabilities Detection in Agile Process Using Graph Method and Deep Neural Network. In Proceedings of the International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC), Bhubaneswar, India, 19–20 November 2022.
30. Ademar Sousa, N.; Ramos, F.; Albuquerque, D.; Dantas, E.; Mirko, P.; Almeida, H.; Perkusich, A. Towards a Recommender System-Based Process for Managing Risks in Scrum Projects. In Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC '23), Tallinn, Estonia, 27–31 March 2023.
31. Odzaly, E.; Greer, D. Lightweight Risk Management in Agile Projects. In Proceedings of the 26th Software Engineering Knowledge Engineering Conference (SEKE), Vancouver, BC, Canada, 3 June 2014. [CrossRef]
32. Odzaly, E.E.; Greer, D.; Stewart, D. Agile Risk Management Using Software Agents. *J. Ambient Intell. Humaniz. Comput.* **2018**, *9*, 823–841. [CrossRef]
33. Tak, A.; Chahal, S. Risk Management in Agile Al/Ml Projects: Identifying and Mitigating Data and Model Risks. *J. Technol. Syst.* **2024**, *6*, 1–18. [CrossRef]
34. Fasching, C.; Gregory, P.; Mitchell, N.; Frowd, C. Risk Identification and Mitigation in Agile Software Re-Engineering: A Case Study. *Commun. IIMA* **2024**, *22*, 44–55. [CrossRef]
35. Prokopenko, T.; Grygor, O.; Prokopenko, V.; Lavdanska, O. Devising a Project Risk Management Method under Scrum Conditions Based on Cognitive Approach. *East.-Eur. J. Enterp. Technol.* **2024**, *5*, 18–26. [CrossRef]